

CSC1006 - Assignment 2

Instructions

Submit your work in the form of a single file onto Blackboard before the deadline. Your submission should include solutions to the theoretical exercises, all the scripts you made use of for addressing the computer-based exercises, and all outputs from your scripts including the figures. You may use a .zip file if you have multiple files or a single .ipynb notebook. Late submissions within 2 days will receive a 50% penalty. Submissions more than 2 days late will not be accepted.

Q1: Parametric and Non-parametric Models Parametric: predefined form of functions, fixed number of parameters
Non-parametric: no fixed assumptions for functional form of data, no fixed number of parameters
Describe the difference of parametric models and non-parametric models. Classify the following ML models into parametric models or non-parametric models.

1. Linear regression. Parametric: learning a fixed set of weights and bias
2. k nearest neighbour. Non-parametric: More data $\rightarrow$ More structure in the model
3. Logistic regression Parametric: Similar to linear regression, just using the sigmoid function instead.
4. Neural network. Parametric: layers and neurons are fixed before training $\rightarrow$ fixed number of weights and biases

Q2: Perceptron Prediction

A perceptron model with two features uses the following formula to make predictions

$$\hat{y} = \text{sign}(w_0x_0 + w_1x_1 + w_2).$$
 (1)

Here $\text{sign}(x)$ is the sign function. It outputs 1 if the input is greater than 0 and outputs -1 otherwise. Suppose $w_0 = 1$, $w_1 = -2$, $w_2 = -1$

1. Make predictions for the following samples (x_0, x_1) : $(3, 0)$, $(1, 1)$. $\hat{y}=1$, $\hat{y}=-1$
2. Plot the decision boundary of the model. Note that decision boundary is the separate-plane or separate-line in feature space (x_0, x_1) , that separate the two classes (one for $\hat{y} = 1$ and one for $\hat{y} = -1$). You may use a Python script for the plotting or plot it by hand. Boundary: $x_0 - 2x_1 - 1 = 0 \rightarrow x_0 = 2x_1 + 1$ If $x_0 > 2x_1 + 1 : x_0 - 2x_1 - 1 > 0 \rightarrow \hat{y} = 1$
 $x_0 < 2x_1 + 1 : x_0 - 2x_1 - 1 < 0 \rightarrow \hat{y} = -1$
3. For the sample $(3, 0)$ within the training set, suppose its correct label is $y = -1$, should the parameters of the perceptron with current parameters values be updated when encountering this data point? if yes, how the parameters (w_0, w_1) should be updated according to the learning rule of perceptron (set learning rate to be 1)? $\alpha=1$
As $y \neq \hat{y}$, the parameters should be updated to avoid misclassification. $\rightarrow$ new value of (w_0, w_1)

Q3: Classification Loss Function $w_{k, \text{new}} = w_k + \alpha(y - \hat{y}) \cdot x_k = \begin{bmatrix} 1 \\ -2 \end{bmatrix} + 1 \cdot (-1 - 1) \begin{bmatrix} 3 \\ 0 \end{bmatrix} = \begin{bmatrix} -5 \\ -2 \end{bmatrix}$ is $(-5, -2)$

The binary cross entropy loss function is frequently used in binary classification problems. For a given prediction $\hat{y}$, whose true value is y , the binary cross entropy loss function takes the following form

$$L(\hat{y}, y) = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y}).$$
 (2)

1. What are the domains (allowed values) of y and $\hat{y}$? y : any value, $\hat{y}$: $0 \leq \hat{y} \leq 1$
2. Plot $L(\hat{y})$ when $y = 0$ and $y = 1$ respectively. You may use a Python script or plot by hand.

3. Explain why minimizing this loss function will drive $\hat{y}$ towards y . $\frac{\partial L}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}$ $\rightarrow y=1$: Gradient $= -\frac{1}{\hat{y}}$ always negative $\rightarrow$ Gradient descend pushes $\hat{y}$ up toward 1
 $\rightarrow y=0$: Gradient $= \frac{1}{1-\hat{y}}$ always positive $\rightarrow$ Gradient descend pushes $\hat{y}$ down toward 0
4. The binary cross entropy loss function penalizes incorrect predictions sharply. Explain why.

The loss function approaches infinity when $\hat{y}$ was a confidently wrong prediction. (Example: $\hat{y}=1$ when $y=0$ or $\hat{y}=0$ when $y=1$)

Q4: Coding: Classify Different Types of Iris

In this task, you are required to build machine learning models to predict whether a given iris flower belongs to a specific type of iris, Versicolour. You can load and initialize the data using the following Python script:

```
from sklearn.datasets import load_iris
X, y = load_iris(return_X_y=True)
y = y == 1 # transform to binary classification
```

1. Divide the dataset into a training set (80%) and a test set (20%). (1/3 Versicolor across both sets)
2. Build a k nearest neighbour model based on the training set, and evaluate the model accuracy on the test set. You may set $k = 3$.
3. Build a logistic regression model for the same task. Compare the performance of the two models.
4. In addition to accuracy, evaluate the precision, recall, and the F_1 score for both models. What deeper insight do you gain from these metrics?
5. (Optional) Further split the dataset into training set, validation set and test set. Choose the best value of k in the KNN model using cross validation.

$k=5$ is the best value among $k=1$ to $k=20$, with CV F_1 score of 0.9198

→ 4. The dataset is imbalanced on the test set (20 non-Versicolour vs 10 Versicolour) → 60% accuracy from linear regression could be achieved by predicting "Not versicolour" most of the time.

Linear regression fails the task as it assumes linear decision boundary, while Iris dataset having non-linear boundaries, causing KNN to achieve perfect classification and make zero errors on both classes.